SecurePass: A Secure RESTful Password Service with C-Based Entropy Analysis

Author: Student **Course:** Software Development / IT Security **Institution:** Hochschule Offenburg (HSO) **Repository:** https://edugit.hs-offenburg.de **Submission:** Project Report & Code Implementation

1. Introduction

Password security remains one of the most critical aspects of modern web applications. Users frequently choose weak, predictable passwords, or reuse the same credentials across multiple services. Furthermore, insecure password storage (such as using plain text or fast legacy hash algorithms like MD5 or SHA-1) allows attackers to compromise user accounts rapidly during a database leak.

The goal of this project is to develop **SecurePass**, a complete and practical password management service that fulfills the following objectives: 1. **Configurable Password Generation:** Create random, unpredictable passwords and passphrases based on user-defined parameters such as length, character sets, and exclusions. 2. **REST API via Flask:** Expose all generation, analysis, and management capabilities through clean, stateless HTTP/JSON endpoints. 3. **Data Persistence via SQLite:** Store user accounts, saved passwords in a vault, and operational security audit logs safely in a local relational database. 4. **PBKDF2 Password Hashing:** Implement cryptographically secure key derivation using Python's standard `secrets` and `hashlib` libraries, protecting stored credentials against offline attacks. 5. **C Programming Integration:** Implement a performance-critical function in native C (calculating Shannon information entropy and character distribution) and call it directly from Python using `ctypes`.

2. Architecture & REST Service Design

The project is structured with a modular architecture to separate concerns between web routing, cryptography, database access, and native C code:

```
[ Web Browser / REST Client ]
             │
             ▼ (HTTP / JSON)
[ Flask Web Server (app.py) ]
   ├── Core Generator (core/generator.py)  --> CSPRNG (secrets)
   ├── PBKDF2 Hasher (core/hasher.py)      --> hashlib & secrets
   ├── C Bridge (core/c_bridge.py)         --> ctypes Foreign Function Interface
   │                                           └── Native C Module (libentropy.dll/.so)
   └── SQLite Database (database/db.py)     --> securepass.db
```

REST API Endpoints

The Flask server (`app.py`) provides the following endpoints:

Method	Endpoint	Description	Status Codes
`GET`	`/api/health`	Service status and C library availability	`200 OK`
`POST`	`/api/generate`	Generates a random password or passphrase with analysis	`200 OK`, `400 Bad Request`
`POST`	`/api/hash`	Hashes a password using PBKDF2 with a random salt	`200 OK`, `400 Bad Request`
`POST`	`/api/verify`	Verifies a password against a hash in constant time	`200 OK`, `400 Bad Request`
`POST`	`/api/analyze`	Evaluates password entropy using the native C library	`200 OK`, `400 Bad Request`
`POST`	`/api/users/register`	Registers a new user with PBKDF2-hashed credentials	`201 Created`, `409 Conflict`
`POST`	`/api/users/login`	Verifies user login credentials	`200 OK`, `401 Unauthorized`
`GET`	`/api/vault`	Retrieves saved password entries	`200 OK`
`POST`	`/api/vault`	Saves a new password entry to SQLite	`201 Created`, `400 Bad Request`
`DELETE`	`/api/vault/`	Deletes a saved password entry by ID	`200 OK`, `404 Not Found`
`GET`	`/api/audit-logs`	Retrieves recent security audit entries	`200 OK`

3. Password Generation with CSPRNG

Standard pseudo-random number generators, such as Python's default `random` module (which uses the Mersenne Twister algorithm), are deterministic and completely insecure for cryptography. If an attacker observes a sequence of outputs, they can reconstruct internal generator state and predict subsequent passwords.

To guarantee cryptographic security, SecurePass exclusively uses Python's `secrets` module: - **`secrets.SystemRandom()` / `secrets.choice()`:** Draws entropy directly from the underlying operating system's cryptographic entropy source (`/dev/urandom` on Linux, `CryptGenRandom` on Windows). - **Configurable Parameters:** - `length`: Integer specifying total password length (default: 16, range: 4 to 256). - Character sets: Toggleable uppercase (`A-Z`), lowercase (`a-z`), digits (`0-9`), and special symbols (`!@#$%^&*...`). - *`exclude_ambiguous`: Filters out characters that look identical in many fonts (`1`, `l`, `I`, `0`, `O`).* - *`custom_exclude`: Blacklists specific characters specified by the caller.* - *`guarantee_each_type`: Guarantees at least one character from every active character pool before shuffling with a Fisher-Yates shuffle.* - ***Passphrase Mode:*** *Generates Diceware-style memorable passphrases (e.g., `Breeze-Canary-Cosmos-Anchor-42`) from a built-in clean wordlist.*

4. PBKDF2 Hashing using `secrets` and `hashlib`

*Storing passwords in plaintext or using fast hash algorithms (like MD5 or SHA-256 alone) is dangerous because attackers can test billions of guesses per second using modern GPUs. To prevent this, SecurePass implements **PBKDF2** (Password-Based Key Derivation Function 2) as recommended by NIST and OWASP.*

4.1 Salt Generation (`secrets`)

A salt is a unique, random string added to the password before hashing. This ensures that two users with identical passwords will have completely different stored hashes, eliminating rainbow table attacks. In `core/hasher.py`, SecurePass generates a fresh 32-byte (256-bit) salt for each password using:

```
salt_bytes = secrets.token_bytes(32)
```

4.2 Key Derivation (`hashlib`)

The key derivation is performed using `hashlib.pbkdf2_hmac`:

```
derived_key = hashlib.pbkdf2_hmac(
    hash_name='sha256',
    password=password.encode('utf-8'),
    salt=salt_bytes,
    iterations=600000,
    dklen=32
)
```

*- **Iterations (Work Factor):** Set to 600,000 iterations for SHA-256 (following OWASP guidelines). This intentionally slows down hash computation, making brute-force attacks computationally impractical. - **Storage Format:** Hashes are saved in a clean modular string format: `pbkdf2:sha256:600000`*

4.3 Constant-Time Verification

During verification, standard string equality (`==`) stops comparing at the first mismatched byte. An attacker measuring response times with high precision could infer characters byte by byte (a timing attack). SecurePass prevents this by using:

```
secrets.compare_digest(candidate_hex, stored_hash_hex)
```

This guarantees constant comparison time regardless of where or whether a mismatch occurs.

5. Calling a Native C Function from Python (`ctypes`)

*To fulfill the requirement of revising C programming by invoking a C function from Python, I implemented a **Password Entropy & Strength Analyzer** in C (`c_modules/entropy_analyzer.c`).*

5.1 What the C Function Does

*The C function evaluates the cryptographic strength of a given password string: 1. **Shannon Information Entropy:** Calculates*

character frequency and computes:

$$H = -\sum_{i=1}^{k} p(x_i) \log_2(p(x_i)) \times \text{length}$$

This measures the actual information density and unpredictability of the string in bits. 2. ***Character Pool Search Space:*** *Counts uppercase letters, lowercase letters, digits, and symbols to determine total combinatorial search space* $E = \text{length} \times \log_2(|R|)$. 3. **Offline Crack-Time Estimation:** Estimates the time required for a high-end GPU cluster (10 billion guesses/second) to brute-force the password. 4. **Composite Strength Score:** Assigns a score (0 to 100) and label ("Very Weak", "Weak", "Moderate", "Strong", "Very Strong").

5.2 C Implementation

In `c_modules/entropy_analyzer.h`, a C structure and exported functions are defined:

```
typedef struct {
    int length;
    int uppercase_count;
    int lowercase_count;
    int digit_count;
    int symbol_count;
    int pool_size;
    double shannon_entropy;
    double pool_entropy;
    double crack_time_seconds;
    int score;
    char strength_label[32];
} PasswordAnalysisResult;

int analyze_password(const char *password, PasswordAnalysisResult *result);
```

5.3 Python Integration with `ctypes`

In `core/c_bridge.py`, Python loads the compiled shared library (`libentropy.dll` on Windows / `libentropy.so` on Linux) and maps the C struct directly using `ctypes`:

```
class PasswordAnalysisResultStruct(ctypes.Structure):
    _fields_ = [
        ("length", ctypes.c_int),
        ("uppercase_count", ctypes.c_int),
        ("lowercase_count", ctypes.c_int),
        ("digit_count", ctypes.c_int),
        ("symbol_count", ctypes.c_int),
        ("pool_size", ctypes.c_int),
        ("shannon_entropy", ctypes.c_double),
        ("pool_entropy", ctypes.c_double),
        ("crack_time_seconds", ctypes.c_double),
        ("score", ctypes.c_int),
        ("strength_label", ctypes.c_char * 32),
    ]

# Load compiled C library
c_lib = ctypes.CDLL("./c_modules/libentropy.dll")
c_lib.analyze_password.argtypes = [ctypes.c_char_p, ctypes.POINTER(PasswordAnalysisResultStruct
```

Python passes the password string as `const char*` and a pointer to the pre-allocated struct. The C function populates the values in native memory, and Python reads the results with zero overhead.

---

6. SQLite Database Persistence

All application data is persisted in a local SQLite3 database (`database/securepass.db`): 1. **`users` Table:** Stores user IDs, unique usernames, PBKDF2 password hashes, unique salts, iteration counts, and creation timestamps. 2. **`vault_items` Table:** Allows users to store generated passwords for specific services (e.g., "HS Offenburg Git", "Email"), with service name, username, password value, category, and notes. 3. **`audit_logs` Table:** Records security events (generation, hashing, verification, login success/failure) with IP addresses and timestamps.

SQL Injection Protection

All database queries use parameterized SQL placeholders (`?`), preventing SQL injection attacks completely:

```
cursor.execute('SELECT * FROM users WHERE username = ?', (username.strip(),))
```

7. Testing and Verification

To ensure reliability, a test suite of **27 automated unit and integration tests** was developed in the `tests/` directory: - **`test_generator.py`:** Tests length boundaries, character set inclusion, ambiguous exclusions, and batch generation. - **`test_hasher.py`:** Tests salt uniqueness, deterministic hash reproducibility, and constant-time verification. - **`test_c_bridge.py`:** Tests dynamic loading of the C shared library and verifies accuracy of C entropy calculations. - **`test_api.py`:** Tests all Flask REST endpoints, status codes, and database CRUD operations.

Test Results:

```
Ran 27 tests in 0.594s - OK (100% Passing)
```

8. Conclusion

The SecurePass project successfully realizes a secure, modular password service meeting all assignment criteria: - Password generation is cryptographically unpredictable via `secrets`. - Passwords are securely hashed with PBKDF2 (600,000 iterations) via `secrets` and `hashlib`. - A native C function was written, compiled into a shared library, and seamlessly integrated into Python using `ctypes`. - Data is safely persisted in SQLite with parameterized queries. - All functionality is exposed through a clean Flask REST API and tested with an automated test suite.